VELoop Rewards

Full-Stack Developer Internship Assignment

Project: SPACE SHOOTER ODYSSEY

Original Space Combat Game + Gamification + Rewarded-Ad Economy
Reference Game: Space Shooter 2
Reference URL:
https://play.google.com/store/apps/details?id=com.game.space.shooter2

Internship Domain: Full-Stack Game Development
Difficulty: Advanced
Project Type: Arcade Shooter + Progression + Ship Upgrades + Reward Economy
Duration: 20–24 days from the assigned date

PART I — PROJECT FOUNDATION

1. Project Objective

Build a production-quality, full-stack browser game named **SPACE SHOOTER ODYSSEY**, inspired by the space-shooter genre and the gameplay concepts of the supplied reference game.

The player controls a spaceship, fights enemy waves, avoids projectiles, defeats bosses, completes missions, earns resources, upgrades their fleet, unlocks abilities, and progresses through increasingly challenging sectors of space.

The central gameplay loop is:

Select Ship → Start Mission → Fight Enemies → Collect Resources → Defeat Boss → Claim Rewards → Upgrade Ship → Unlock New Missions.

The project has two equally important engineering objectives:

1. **Game Engineering:** Create enjoyable, responsive, replayable arcade combat.
2. **Game Economy Engineering:** Build a secure, controlled, measurable, and sustainable reward economy suitable for VELoop Rewards.

The objective is not to maximize the number of advertisements. The objective is to provide optional rewarded advertisements at meaningful moments while preserving gameplay quality, player trust, and economic sustainability.

The completed game must be a functioning product, not merely a collection of screens.

2. Mandatory — Play the Reference Game First

Before writing code, open the reference game:

https://play.google.com/store/apps/details?id=com.game.space.shooter2

Install and play it if your device supports it. If installation is unavailable, inspect the official listing and available gameplay footage.

Study the following areas:

- Main menu and game navigation.
- Spaceship selection and customization.
- Ship movement and firing.
- Enemy formations and projectile patterns.
- Boss battles.
- Mission and stage progression.
- Power-ups and special abilities.
- Health, shields, and player defeat.
- Ship upgrades and resource costs.
- Mission completion and reward screens.
- Daily activities and progression systems.
- Advertisement placements and reward opportunities.
- Audio, visual effects, animations, and responsiveness.

Create a brief research document describing the observed gameplay concepts, what you plan to implement, and how your game will differ.

The reference is for research only. Do not assume that every feature listed in this assignment exists in the reference game.

3. Important — Do Not Copy the Reference Game

Do not copy:

- Original game branding or logo.
- Original title artwork.
- Proprietary spaceships or enemies.
- Original level layouts.
- Exact interface screenshots.
- Original audio, music, or animations.
- Original source code or APK assets.

- Proprietary mission names or artwork.

Create your own:

- Game identity and logo.
- Spaceship designs.
- Enemy types.
- Galaxy and sector artwork.
- Boss designs.
- Mission layouts.
- Power-up icons.
- User interface.
- Sounds and music.
- Reward system presentation.
- Original gameplay variations.

You may study general space-shooter mechanics, but the final implementation must be independently developed.

4. Dynamic Game Name — Mandatory

The game name must be configurable through environment variables. It must not be repeatedly hardcoded across the application.

Frontend .env:

```
VITE_GAME_NAME=SPACE SHOOTER ODYSSEY
VITE_API_URL=http://localhost:5000/api
```

Backend .env:

```
GAME_NAME=SPACE SHOOTER ODYSSEY
MONGODB_URI=
JWT_SECRET=
```

Create a centralized frontend configuration module:

src/config/gameConfig.js

Example:

```
export const GAME_NAME =
  import.meta.env.VITE_GAME_NAME || "SPACE SHOOTER ODYSSEY";
```

The backend must obtain its configured game name from its environment configuration as well.

Use the centralized configuration for:

- Landing page.
- Navigation.
- Game screen.
- Mission selection.
- Ship hangar.
- Player dashboard.
- Reward center.
- Wallet.
- Achievements.
- Leaderboard.
- Settings.
- Help pages.
- Loading screens.
- Game-over screens.

- Mission-completion screens.
- Browser title and metadata.
- SEO metadata.
- Open Graph metadata.
- PWA metadata, if implemented.
- Admin screens.
- Relevant notification and email templates.

Create .env.example files for all required variables. Add actual environment files and credentials to .gitignore.

Evaluation test: An evaluator may change VITE_GAME_NAME to GALAXY COMMANDER or STAR PILOT ODYSSEY. After rebuilding, the new title must appear throughout the application without manual component edits.

There must be one source of truth for the game name.

PART II — GAMEPLAY ENGINE

5. Core Gameplay

Implement a playable, responsive space-shooter game.

The player must be able to:

1. Select an unlocked spaceship.
2. Start an available mission.
3. Move around the playable area.
4. Fire weapons.
5. Avoid enemy projectiles.
6. Destroy enemies.
7. Collect eligible in-game pickups.
8. Activate power-ups.
9. Survive enemy waves.
10. Defeat a boss where the mission requires one.
11. Complete the mission or lose the run.
12. Review the mission result.
13. Receive eligible rewards.
14. Upgrade their ship and continue progression.

The gameplay must have actual mechanics. A spaceship image moving across the screen without collision detection, enemies, combat, and mission objectives does not satisfy this requirement.

6. Game Rendering and Technology

Recommended options include:

- Phaser for a structured 2D game engine.
- Canvas for a custom lightweight rendering system.
- PixiJS for GPU-accelerated 2D rendering.
- React for the application interface, navigation, dashboards, and overlays.

A hybrid architecture is encouraged: use an appropriate game engine for combat and React for the surrounding application.

The intern may select a different stack if the choice is documented and the required functionality is delivered.

Avoid building the entire game using ordinary DOM elements if the resulting combat becomes inefficient or difficult to control.

7. Player Movement and Controls

Desktop controls

Implement:

- WASD or arrow-key movement.
- A configurable firing control if firing is not automatic.
- Power-up activation.
- Pause and resume.
- Restart after the run ends.
- Keyboard focus handling for menus and dialogs.

Mobile controls

Implement:

- Touch-based movement.
- Drag or virtual joystick controls.
- A clearly visible special-ability button.
- A pause control.
- Responsive firing behaviour.

Choose the control method best suited to the game engine, but document it.

Movement requirements

- Movement must be responsive.
- The spaceship must remain inside the permitted play area.
- Simultaneous movement inputs must behave predictably.
- Controls must not interfere with browser scrolling during active gameplay.
- Movement must remain consistent across different frame rates.
- The game must pause or safely handle tab visibility changes where appropriate.

8. Player Health, Damage, and Defeat

Implement:

- Configurable maximum health.
- Damage from enemy projectiles and collisions.
- A visible health indicator.
- Shield status where applicable.
- Hit animations and feedback.

- Temporary post-damage invulnerability where appropriate.
- A defined defeat condition.
- A checkpoint or run state suitable for the revive feature.

Damage and health changes must affect the actual game state, not only a displayed number.

A defeated player must not continue receiving normal mission rewards unless the game rules explicitly allow an eligible partial reward.

9. Enemy System

Implement multiple enemy types with different gameplay roles.

Minimum enemy categories:

Scout Fighter

- Basic movement.
- Low durability.
- Simple projectile attack.

Interceptor

- Faster movement.
- Aggressive attack patterns.
- Lower durability than heavily armored enemies.

Armored Cruiser

- Higher health.
- Slower movement.
- Stronger attacks.

Long-Range Gunship

- Maintains distance from the player.
- Fires targeted or patterned projectiles.

Elite Enemy

- More complex movement.
- Higher durability.
- Special attacks.
- Better mission reward value.

Every enemy must have configurable properties such as:

- Unique identifier.
- Health.
- Movement speed.
- Damage.
- Firing interval.
- Projectile pattern.
- Score value.
- Drop eligibility.
- Spawn conditions.
- Difficulty scaling.

Use shared enemy configuration and reusable behaviour modules. Do not implement all enemies as the same object with different images.

10. Enemy Waves and Spawn System

Create a wave-based encounter system.

Each mission should define:

- Wave count.
- Enemy types.
- Spawn positions.
- Spawn intervals.
- Wave completion conditions.
- Difficulty parameters.
- Boss conditions, if applicable.

Difficulty should increase through more demanding enemy patterns, increased durability, new enemy combinations, and additional objectives.

Do not make later levels difficult only by increasing enemy health to extreme values.

Spawn rates and projectile counts must remain performant on mobile devices.

11. Boss Battles

Implement at least **five original boss encounters** across the campaign.

Each boss must have:

- A unique visual design.
- A boss health bar.
- Defined attack patterns.
- Telegraphing before powerful attacks.
- Multiple phases for major bosses.
- Distinct movement or positioning behaviour.
- A defeat animation.
- A mission-result state.
- A defined reward calculation.

Example original bosses:

1. **The Iron Warden** — a heavily armored combat vessel.
2. **Nebula Serpent** — a fast-moving alien craft.
3. **Solar Devourer** — a large energy-based enemy.
4. **The Rift Carrier** — a carrier that deploys smaller enemies.
5. **Void Emperor** — a final-sector boss with several attack phases.

These are original design suggestions. You may improve the names and concepts.

Bosses must be challenging but fair. Powerful attacks should provide a reasonable opportunity to react or evade.

12. Projectiles and Collision Detection

Implement proper collision handling between:

- Player and enemy projectiles.
- Player and enemies.
- Player projectiles and enemies.
- Player projectiles and boss hitboxes.
- Power-ups and the player.
- Collectible pickups and the player.
- Special abilities and eligible targets.

Define damage rules and projectile ownership. The player's projectiles must not damage the player unless an explicitly designed mechanic requires it.

Avoid repeatedly applying damage every frame while two objects overlap unless continuous damage is intended.

Use suitable collision groups, object pooling, or other performance techniques as needed.

13. Combat Pickups

Implement collectible objects that may appear during gameplay.

Examples:

- Credit Cache.
- Energy Core.
- Temporary Shield.
- Weapon Upgrade.
- Rapid-Fire Module.
- Repair Module.

Every pickup must have defined eligibility and effects.

Important economy rule: Visual pickup collection must not automatically create unlimited redeemable or platform-backed rewards. Server-backed currency rewards must follow authoritative session and reward validation.

PART III — MISSIONS AND PROGRESSION

14. Campaign and Sector System

Create a multi-sector campaign.

Suggested original sectors:

1. Orion Frontier.
2. Crimson Nebula.
3. Frozen Expanse.
4. Rift Territory.
5. Void Citadel.

Each sector should have its own background, enemy composition, mission objectives, and visual identity.

Sector progression must be data-driven. Do not create a separate React page for every mission.

15. Minimum Mission Count

Implement **at least 50 playable missions**.

Suggested difficulty distribution:

Missions	Difficulty	Purpose
1–10	Tutorial / Beginner	Introduce controls, enemies, and pickups
11–20	Easy	Build combat familiarity
21–30	Medium	Introduce stronger enemies and new patterns
31–40	Hard	Require upgrades and tactical power-up use
41–50	Expert	Combine advanced enemy waves and boss encounters

At least five missions must contain functional boss encounters.

A mission counts as playable only when its level data, enemy configuration, objectives, completion rules, and progression conditions are functional.

Do not create 50 mission cards that all launch the same hardcoded encounter.

16. Data-Driven Mission Configuration

Store mission definitions in structured data.

Example:

```
{
 id: "orion-01",
 sectorId: "orion-frontier",
 sequence: 1,
 difficulty: "easy",
 objective: {
  type: "destroy_enemies",
  requiredCount: 15
 },
 playerShipRequired: "interceptor",
 waves: [
  {
   id: 1,
   enemies: [
    {
     type: "scout",
     count: 5
    }
   ]
  }
 ],
 bossId: null,
 rewards: {
  credits: 100,
  xp: 50
 },
 unlockRequirements: []
```

}

The example is illustrative. Expand the structure to support the actual game engine and requirements.

Mission data should support:

- Enemy waves.
- Boss encounters.
- Objectives.
- Difficulty.
- Unlock requirements.
- Reward configuration.
- Mission modifiers.
- Completion conditions.
- Optional challenge goals.

17. Mission Objectives

Implement multiple objective types.

Minimum objectives:

- Destroy all required enemies.
- Survive a specified duration.
- Defeat a designated boss.
- Protect a target for a defined period.
- Destroy a specified enemy type.
- Collect required mission items.
- Complete an encounter with limited damage.
- Complete a mission within a time target.

Each objective must have its own validation and progress tracking.

Do not make every mission a simple requirement to destroy all enemies.

18. Mission Unlocking

Initially, only the first mission should be unlocked.

Completing a mission should unlock the next eligible mission according to the campaign rules.

The server must validate unlock eligibility. A user must not be able to unlock every mission by modifying local storage or a frontend variable.

Support:

- Locked missions.
- Available missions.
- Completed missions.
- Best results.
- Replay of eligible missions.
- Sector unlock conditions.

19. Mission Results, Score, and Stars

Create a detailed result screen.

Display:

- Mission result.
- Final score.
- Enemies defeated.
- Bosses defeated.
- Accuracy where measurable.
- Damage taken.
- Mission duration.
- Power-ups used.
- Credits earned.
- XP earned.
- Stars awarded.
- New unlocks.

Implement a three-star rating system based on documented performance criteria.

For example:

- One star: complete the mission.
- Two stars: meet a performance target.
- Three stars: meet a higher performance target.

Use configurable conditions such as accuracy, time, damage taken, or optional objectives.

Do not award random scores unrelated to actual gameplay.

20. Mission Session Lifecycle

When a player starts a mission, create a game session with a unique identifier.

The lifecycle should support:

1. Session initialization.
2. Ship and mission validation.
3. Initial state creation.
4. Combat.
5. Objective tracking.
6. Pause and resume handling.
7. Mission completion or defeat.
8. Final result calculation.
9. Reward eligibility validation.
10. Reward transaction processing.
11. Session finalization.

A finalized session must not be reused to claim the same completion reward repeatedly.

21. Pause, Restart, and Resume

Implement:

- Pause menu.
- Resume.
- Restart mission.
- Exit to the hangar or campaign.
- Confirmation before abandoning a mission.
- Appropriate state cleanup.

A restart must reset the current run correctly. It must not duplicate previously earned rewards or create extra mission sessions that bypass reward restrictions.

PART IV — SPACESHIP FLEET AND UPGRADES

22. Ship Hangar

Create a dedicated ship-hangar interface.

Players should be able to:

- Browse ships.
- Inspect ship attributes.
- Select an owned ship.
- View locked ships.
- Review unlock conditions.
- Upgrade eligible ships.
- Preview weapon and defensive characteristics.

The hangar must reflect authoritative player ownership and upgrade state.

23. Required Ship Classes

Implement at least five original ship classes.

Interceptor

Fast movement, balanced firepower, and moderate durability.

Striker

High damage output and weaker defensive capabilities.

Guardian

Higher health, defensive strengths, and slower movement.

Support Craft

Utility-focused abilities, improved cooldowns, or enhanced pickup collection.

Experimental Ship

Advanced capabilities unlocked through difficult missions or defined progression milestones.

Each class must have meaningful statistical and gameplay differences.

24. Ship Upgrade Attributes

Implement upgrades for:

- Weapon damage.
- Firing rate.
- Maximum health.
- Movement speed.
- Shield capacity.
- Special-ability cooldown.
- Projectile characteristics.

Each upgrade must have:

- Current level.
- Maximum level.
- Upgrade cost.
- Attribute change.
- Unlock requirements.
- Purchase eligibility.
- Clear feedback.

Upgrades must modify the actual gameplay state.

25. Upgrade Economy

Use the appropriate in-game currency for each purchase.

The upgrade process must:

1. Authenticate the player.
2. Retrieve the authoritative ship state.
3. Validate the upgrade and maximum level.
4. Determine the correct server-controlled price.
5. Verify sufficient currency.
6. Deduct the currency atomically.
7. Apply the upgrade.
8. Record a wallet transaction.
9. Return the updated authoritative state.

Prevent duplicate spending, negative balances, and unauthorized ship upgrades.

PART V — POWER-UPS AND COMBAT ABILITIES

26. Mandatory Power-Ups

Implement at least the following power-ups as real gameplay mechanics.

Shield Pulse

Temporarily protects the player from incoming damage.

Requirements:

- Defined duration.
- Visual shield effect.

- Active-state indicator.
- Cooldown.
- Defined interaction with boss attacks.

Rapid Fire

Temporarily increases the firing rate.

Requirements:

- Configurable multiplier.
- Defined duration.
- Visual and audio feedback.
- Correct restoration of normal firing speed.

Spread Shot

Fires multiple projectiles in a defined spread.

Requirements:

- Projectile-angle configuration.
- Projectile limits.
- Damage rules.
- Upgrade compatibility.

Homing Missiles

Launches limited missiles that track eligible targets.

Requirements:

- Target-selection rules.
- Missile count.
- Damage values.
- Lifetime.
- Cooldown or inventory consumption.

EMP Blast

Disables or damages eligible enemies within a defined radius.

Requirements:

- Area-of-effect configuration.
- Eligible target rules.
- Duration or damage.
- Visual range indicator.

Repair Drone

Restores a limited amount of player health.

Requirements:

- Defined repair amount.
- Maximum-health validation.
- Cooldown or consumption rules.

Overdrive

Temporarily improves selected combat statistics.

Requirements:

- Defined multipliers.
- Duration.
- Stacking rules.
- Expiration handling.

Emergency Warp

Moves the player to a safer position or grants a short defensive window.

Requirements:

- Defined activation rules.
- Cooldown.
- Collision handling.
- Abuse prevention.

27. Power-Up Configuration

Create a centralized configuration containing:

- Power-up identifier.
- Name.
- Description.
- Icon.
- Effect type.
- Duration.
- Cooldown.
- Maximum uses.
- Unlock conditions.
- Acquisition method.
- Currency cost, where applicable.
- Rewarded-ad eligibility.
- Stacking rules.

Do not scatter power-up values throughout unrelated components.

28. Power-Up Inventory

Implement an inventory where appropriate.

Players must be able to:

- View available power-ups.
- Inspect their effects.
- See cooldowns and remaining uses.
- Acquire eligible items.
- Activate available items.
- Understand when an item is unavailable.

Power-up consumption and server-backed purchases must be validated. Client-side inventory manipulation must not create free paid items.

29. Power-Up Balance

Define balancing rules for all power-ups.

No ability should remain permanently active unless that behaviour is explicitly intended.

Prevent:

- Unlimited shield activation.
- Unlimited healing.
- Infinite missiles.
- Stacking that creates excessive damage.
- Cooldown bypass through repeated requests.
- Reward claims triggered by ability activation.

Use server-controlled configuration for important economy parameters and define how active combat effects are validated.

PART VI — VELoop REWARD ECONOMY

30. Economy Design — Mandatory

VELoop Rewards is a rewards and engagement platform. Therefore, the game must have a carefully designed economy.

The economic flow is:

Player Activity → Eligible Ad Opportunity → Verified Ad Event → Controlled Reward → Engagement and Retention.

The objective is not to distribute unlimited coins. Rewards must be useful, measurable, controlled, auditable, and sustainable.

31. Currency Types

Implement separate currency types.

Credits

The standard gameplay currency.

Used for:

- Ship upgrades.
- Standard unlocks.
- Selected power-ups.
- Gameplay utilities.

Energy Cores

A rarer progression currency.

Used for:

- Advanced ship unlocks.
- High-level upgrades.

- Special gameplay items.

XP

Used for player progression and level unlocks.

XP must be distinct from spendable currency.

VELoop Reward Coins

A separate balance for platform-eligible rewards, subject to VELoop's actual rules.

Do not automatically equate Credits, Energy Cores, XP, and VELoop Reward Coins.

Do not imply that ordinary gameplay currencies have cash value or can be redeemed externally unless the platform explicitly supports that functionality.

32. Example Economy Values

Create a centralized configuration with initial balancing values.

Activity	Illustrative reward
Complete a standard mission	100 Credits
Destroy a normal enemy	2 Credits
Defeat an elite enemy	10 Credits
Defeat a boss	250 Credits
Complete a daily mission	150 Credits
Earn a three-star rating	50 Credits
Reach a progression milestone	Configurable XP and Credits
Verified eligible rewarded ad	Configurable reward

These are starting values, not final commercial rates.

Adjust values based on mission duration, difficulty, upgrade costs, player progression, and observed advertising economics.

Do not award the same event twice through overlapping callbacks or repeated requests.

33. Economy Features

Implement:

- Wallet balances.
- Wallet transaction history.
- Reward transaction history.
- Mission-completion rewards.
- Boss rewards.
- Daily mission rewards.

- Achievement rewards.
- Eligible rewarded-ad rewards.
- Upgrade spending.
- Reward caps.
- Cooldowns.
- Eligibility checks.
- Duplicate-transaction protection.
- Centralized economy configuration.
- Administrative audit data.

Every transaction must have a clear source, amount, type, timestamp, and reference to the originating event.

34. Economy Configuration

Create a central economy configuration module.

Example:

```
export const ECONOMY_CONFIG = {
  missionCompletionCredits: 100,
  eliteEnemyCredits: 10,
  bossCompletionCredits: 250,
  threeStarBonusCredits: 50,

  rewardedAds: {
    enabled: true,
    reviveEnabled: true,
    doubleScoreEnabled: true,
    doubleCreditsEnabled: true,
    extraShieldEnabled: true,

    maxRevivesPerMission: 1,
    maxDoubleScorePerMission: 1,
    maxDoubleCreditsPerMission: 1,

    maxRewardedAdsPerDay: 10,
    maxRewardedAdsPerSession: 5
  }
};
```

These values are examples and must be tuned.

The backend must be authoritative for rewards, prices, eligibility, and limits.

35. Server-Controlled Economy

Provide an endpoint such as:

```
GET /api/economy/config
```

The server should return the currently enabled economy options and relevant limits.

The frontend must not be able to change authoritative reward values by editing JavaScript variables.

Allow authorized administrators to adjust supported configuration values without requiring a full frontend rebuild.

Configuration changes must be validated, logged, and applied consistently.

36. Wallet Ledger

Do not simply update a user balance without recording why the change occurred.

Record wallet transactions with fields such as:

- User ID.
- Game ID.
- Transaction type.
- Amount.
- Balance before.
- Balance after.
- Source.
- Reference ID.
- Status.
- Timestamp.

Possible transaction types:

- CREDIT
- DEBIT
- MISSION_REWARD
- BOSS_REWARD
- AD_REWARD
- UPGRADE_PURCHASE
- BOOSTER_PURCHASE
- DAILY_BONUS
- REVERSAL

The wallet and transaction ledger must remain consistent.

37. Reward Transaction Ledger

Maintain a separate record for reward events where appropriate.

Suggested fields:

- User ID.
- Game ID.
- Session ID.
- Mission ID.
- Reward type.
- Amount.
- Multiplier.
- Source.
- Ad provider.
- Ad event ID.
- Verification status.
- Creation timestamp.

The transaction ledger must allow an administrator to investigate duplicate events, rejected claims, and unexpected reward activity.

38. Atomic Currency Transactions

Currency changes must be atomic.

For an upgrade purchase:

1. Load the authoritative player state.
2. Validate ownership and upgrade eligibility.
3. Determine the server-controlled price.
4. Confirm sufficient currency.
5. Deduct the currency.
6. Apply the upgrade.
7. Record the transaction.
8. Commit the operation.

If an operation fails, it must not leave the player with a deducted balance and no upgrade.

Use database transactions where supported or another appropriate atomic consistency strategy.

39. Idempotency

Idempotency is mandatory for reward processing.

If the same verified ad event is received twice, the player must receive the reward only once.

Use unique event identifiers or equivalent transaction-level protection.

Also prevent:

- Duplicate mission-completion rewards.
- Duplicate daily claims.
- Repeated reward requests.
- Replayed mission sessions.
- Duplicate achievement rewards.

PART VII — REWARDED ADVERTISING

40. Rewarded-Ad Strategy

Prioritize optional rewarded advertisements rather than disruptive interruptions.

Potential placements include:

1. Revive after mission failure.
2. Double score after mission completion.
3. Double Credits after mission completion.
4. Temporary Shield or another eligible power-up.
5. Additional mission reward.
6. Optional daily reward bonus.

The player must always have a meaningful choice.

Core gameplay must not require watching an advertisement.

41. Revive System

When the player is defeated, display:

MISSION FAILED

Continue from your last eligible checkpoint?

- **Watch Ad & Revive**
- **Restart Mission**
- **Return to Hangar**

A successful verified rewarded ad may allow the player to continue from the last valid checkpoint.

The revive flow must:

1. Save a valid checkpoint or run state.
2. Create a unique revive opportunity.
3. Verify the mission session and player eligibility.
4. Start the rewarded advertisement.
5. Verify the provider's completion event.
6. Confirm the revive limit has not been reached.
7. Restore the eligible checkpoint.
8. Record the revive transaction.
9. Resume the mission.

The server must validate the revive count, session ownership, current game state, and reward eligibility.

Example configuration:

```
MAX_REVIVES_PER_MISSION = 1;
```

A player must not obtain unlimited revives by repeatedly reopening the dialog.

42. Double Score

After a successful mission, show the result and an optional double-score offer.

Example:

```
MISSION COMPLETE

Final Score: 4,500

[CLAIM SCORE]
[WATCH AD → DOUBLE SCORE]
```

If the player completes the eligible rewarded advertisement, apply the permitted multiplier.

Requirements:

- Maximum one double-score reward per eligible mission, unless explicitly configured otherwise.
- The server must calculate the final eligible score.
- The event must be linked to the correct mission session.
- The same event cannot be processed twice.
- A client-submitted score multiplier must not be trusted.

Score multipliers must not create an unlimited path to platform-backed rewards.

43. Double Credits

After a mission, display the eligible Credit reward.

Example:

MISSION REWARDS

You earned 100 Credits.

[CLAIM 100]
[WATCH AD → GET 200]

After verified ad completion, the backend determines the actual eligible reward.

Requirements:

- Configurable multiplier.
- Maximum one double-Credits claim per eligible mission by default.
- Daily and session limits.
- Idempotent processing.
- Wallet transaction.
- Updated authoritative balance.

Do not trust a request such as { "credits": 5000 } from the browser.

44. Extra Shield or Power-Up Reward

Provide an optional rewarded-ad offer for an eligible temporary defensive ability or power-up.

For example:

EMERGENCY SHIELD

Need extra protection?

[WATCH AD → GET SHIELD]
[NOT NOW]

The backend must verify eligibility and enforce acquisition limits.

The power-up must have a defined effect, duration, and stacking rule.

45. Additional Mission Reward

Where appropriate, offer an optional ad-based bonus after mission completion.

This must be separate from the standard mission reward. The player must receive

the standard reward even if they decline the ad, provided they are otherwise eligible.

Do not allow the same mission result to trigger multiple bonus claims.

46. Rewarded-Ad State Machine

Use explicit states such as:

- OFFERED
- REQUESTED
- AD_STARTED
- AD_COMPLETED
- VERIFYING
- VERIFIED
- GRANTED
- REJECTED
- EXPIRED

Keep advertisement state separate from ordinary gameplay state.

A frontend button click is not proof of a completed advertisement.

47. Ad Opportunity Model

Create an ad-opportunity model containing:

- User ID.
- Game session ID.
- Mission ID.
- Placement type.
- Provider.
- Status.
- Expiration time.
- Unique event identifier.
- Creation timestamp.

Supported placements should include:

- REVIVE
- DOUBLE_SCORE
- DOUBLE_CREDITS
- POWER_UP
- DAILY_BONUS

Use centralized placement identifiers rather than arbitrary strings across the application.

48. Ad Provider Abstraction

Do not tightly couple the game to a single advertising provider.

Suggested structure:

```
server/
  src/
    services/
      ads/
        adService.js
        rewardedAdService.js
        adProviderAdapter.js
```

Define an adapter interface that supports the selected provider's actual capabilities.

For local development, provide a clearly labelled simulated-ad adapter.

A simulated adapter may test the user interface and state transitions, but it must not be described as a production ad integration.

When a real provider is configured, use its supported trusted callback or server-side verification mechanism.

49. Failed Advertisements

If an ad fails to load:

- Show a clear error message.
- Allow the player to retry.
- Do not grant the reward.
- Do not consume an unused revive.
- Do not deduct currency.
- Do not mark the ad opportunity as successfully completed.

If the player closes an ad before the provider confirms completion, do not grant the reward.

If a callback is duplicated, grant the reward only once.

50. Ad Frequency and Reward Caps

Create configurable limits for:

- Rewarded ads per session.
- Rewarded ads per hour.
- Rewarded ads per day.
- Minimum time between ads.
- Revives per mission.
- Double-score claims per mission.
- Double-Credits claims per mission.
- Power-up acquisition.
- Daily bonus claims.

Example:

```
const AD_LIMITS = {
  maxRewardedPerSession: 5,
```

```
  maxRewardedPerHour: 8,
  maxRewardedPerDay: 10,
  minSecondsBetweenRewardedAds: 30
};
```

These are example limits, not final business decisions.

Do not show an advertisement:

- During active combat.
- Immediately after every button click.
- In a rapid loop after closing another ad.
- Before every restart.
- As an unavoidable step in the core gameplay.
- At a frequency that makes the game unusable.

If interstitial advertisements are implemented, use a configurable cooldown and do not place them in the middle of combat or immediately after a rewarded ad.

51. Economy Sustainability

Every reward has an economic cost.

The design must account for:

- Expected advertising revenue.
- Reward allocation.
- Maximum reward liability.
- User engagement.
- Reward frequency.
- Conversion from ad opportunity to completed ad.
- Currency sinks such as upgrades and power-ups.

The intern does not need access to confidential advertising payout data. The architecture must allow reward values to be tuned when actual monetization data becomes available.

Do not implement:

- Unlimited Credits.
- Unlimited revives.
- Unlimited double rewards.
- Client-authorized reward grants.
- Unlimited power-up claims.
- A wallet stored only in local storage.
- Rewards for unverified ad completion.

PART VIII — ENGAGEMENT AND RETENTION

52. Daily Missions

Implement at least three daily mission templates, such as:

- Destroy 30 enemies.
- Complete two missions.
- Defeat one elite enemy.
- Use a power-up three times.
- Complete a mission without losing all health.

Requirements:

- Mission progress.
- Completion status.
- Reward configuration.
- Defined reset rules.
- Server-side claim validation.
- Duplicate-claim prevention.

Daily missions should encourage different gameplay styles.

53. Daily Login Streak

Create a daily login reward system.

Track:

- Current streak.
- Last eligible claim time.
- Claim history.
- Next reward.
- Streak reset conditions.

Display a clear reward calendar or streak indicator.

Use consistent server-side date and timezone rules. Changing the device clock must not allow repeated claims.

Avoid exponential rewards that destabilize the economy.

54. Achievements and Badges

Implement at least ten achievements.

Suggested achievements:

1. **First Contact** — defeat the first enemy.
2. **Fleet Recruit** — unlock a second ship.
3. **Elite Hunter** — defeat a specified number of elite enemies.
4. **Boss Breaker** — defeat the first boss.
5. **Sector Explorer** — unlock a new sector.
6. **Ace Pilot** — complete a mission with a defined accuracy target.
7. **Shield Specialist** — successfully use the shield ability.
8. **Power Collector** — use several power-up types.
9. **Fleet Commander** — unlock multiple ships.
10. **Void Conqueror** — complete the final campaign mission.

Each achievement must have defined completion conditions and must be awarded only once.

55. Player Level and XP

Implement a progression system that awards XP for eligible activities.

Include:

- Player level.
- Current XP.
- XP required for the next level.
- Progress bar.
- Level-up notification.
- Unlock milestones.
- Configurable progression rewards.

XP calculation and progression updates must be validated by the server.

Do not accept arbitrary XP amounts directly from the client.

56. Mission Streaks

Track consecutive eligible mission completions.

Provide:

- Current streak.
- Streak progress.
- Defined reset rules.
- Optional milestone rewards.
- Duplicate-reward prevention.

Streak rewards must be configurable and must not produce unlimited currency.

57. Supply Drop

Create an optional supply-drop feature.

Possible rewards include:

- Credits.
- Energy Cores.
- Temporary power-ups.
- Ship cosmetics.
- Eligible progression items.

Requirements:

- Claim eligibility.
- Cooldowns.
- Configurable reward tables.
- Defined probabilities where random outcomes are used.
- Server-side selection and grant processing.
- Claim history.
- Limits on repeat claims.

Do not describe a random reward as guaranteed.

58. Limited-Time Events

Implement at least one functional demonstration event.

The event configuration should contain:

- Event identifier.
- Name and description.
- Start and end time.
- Eligibility rules.
- Objectives.
- Progress tracking.
- Reward configuration.
- Completion status.

Example: a limited-time event that asks players to destroy a specified number of elite enemies across eligible missions.

The backend must validate the event period and eligible progress.

59. Leaderboard

Create a leaderboard based on validated mission performance.

Display:

- Player rank.
- Display name.
- Score.
- Missions completed or another defined metric.
- Player's own rank.
- Pagination or limited result sets.

Define tie-breaking rules.

The backend must validate eligible score submissions and prevent duplicate or manipulated results.

Weekly and monthly leaderboards are optional extensions.

PART IX — AUTHENTICATION AND PLAYER DATA

60. Authentication

Implement:

- Registration.
- Login.
- Logout.
- Current-user retrieval.
- Protected routes.
- Secure password hashing.
- Appropriate session or token handling.

Suggested endpoints:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET /api/auth/me

Use secure cookies where appropriate, or implement a well-designed token strategy.

Never store plaintext passwords.

Validate input and return appropriate error messages without exposing secrets.

61. Player Dashboard

Create a player dashboard showing:

- Player level.
- Current XP.
- Selected spaceship.
- Unlocked ships.
- Missions completed.
- Current sector.
- Total score.
- Best score.
- Achievements.
- Credits.
- Energy Cores.
- Eligible VELoop Reward Coins.
- Daily streak.
- Rewarded-ad activity.

The displayed values must be based on persisted data or clearly labelled demonstration data.

62. Game History

Persist eligible mission history.

Store:

- Session ID.
- Mission ID.
- Selected ship.
- Result.
- Score.
- Enemies defeated.
- Bosses defeated.
- Mission duration.
- Damage taken.
- Power-ups used.
- Stars awarded.
- Credits earned.
- Completion time.

History must be associated with the correct player.

63. Persistent Player Progress

Persist:

- Unlocked missions.
- Completed missions.
- Best mission results.
- Selected ship.
- Ship upgrades.
- Inventory.

- XP and level.
- Achievements.
- Daily mission progress.
- Daily claim history.
- Wallet balances.
- Transaction history.

The player must not lose legitimate progress merely because the browser is refreshed.

PART X — SERVER VALIDATION AND ANTI-CHEAT

64. Game Session Model

When a mission starts, create a session record.

Suggested fields:

```
{
 sessionId,
 userId,
 gameId,
 missionId,
 selectedShipId,
 status,
 startTime,
 endTime,
 initialState,
 checkpointState,
 result
}
```

Use unique session identifiers and validate session ownership.

65. Score Validation

The backend must validate mission results before granting rewards or updating the leaderboard.

Validate:

- Mission ID.
- Session ownership.
- Session status.
- Selected ship eligibility.
- Mission completion conditions.
- Score bounds.
- Enemy counts.
- Boss completion.
- Mission duration.
- Power-up usage.
- Reward eligibility.
- Duplicate completion.
- Previous reward claims.

The browser must never be the sole authority for score or reward decisions.

For stronger integrity, use a server-authoritative simulation or a validated event stream where appropriate. Document the limitations of client-simulated combat and implement reasonable anti-cheat controls for the chosen architecture.

66. Anti-Cheat Requirements

Detect or reject:

- Impossible scores.
- Impossible completion times.
- Invalid mission IDs.
- Duplicate session submissions.
- Replayed requests.
- Duplicate ad events.
- Invalid reward amounts.
- Unauthorized ship upgrades.
- Negative currency balances.
- Repeated daily claims.
- Unauthorized inventory changes.
- Attempts to modify another player's data.

Use rate limiting for sensitive endpoints.

Do not claim that a basic browser-side check prevents all cheating. Document the actual protection provided.

PART XI — DATABASE AND API DESIGN

67. Recommended Database Models

Use a maintainable data model covering:

- User
- PlayerProfile
- GameSession
- Mission
- PlayerMissionProgress
- Ship
- PlayerShip
- PowerUp
- PlayerInventory
- EconomyConfig
- WalletTransaction
- RewardTransaction
- AdOpportunity
- DailyMission
- PlayerDailyMission
- DailyClaim
- Achievement
- PlayerAchievement
- GameEvent
- LeaderboardEntry

These are suggested model boundaries. Combine or separate collections where appropriate, provided consistency and query requirements are satisfied.

Create appropriate indexes and unique constraints for session IDs, transaction identifiers, and other fields requiring uniqueness.

68. Required API Groups

Document REST endpoints for the following areas.

Player

GET /api/player/profile
GET /api/player/progress
GET /api/player/inventory
GET /api/player/history

Missions

GET /api/missions
GET /api/missions/:missionId
POST /api/games/start
POST /api/games/:sessionId/complete
POST /api/games/:sessionId/claim
GET /api/games/history

Ships

GET /api/ships
GET /api/ships/owned
POST /api/ships/:shipId/select
POST /api/ships/:shipId/upgrade
POST /api/ships/:shipId/unlock

Power-Ups

GET /api/powerups
GET /api/player/inventory
POST /api/powerups/:powerUpId/acquire
POST /api/games/:sessionId/powerups/use

Economy

GET /api/economy/config
GET /api/wallet/balance
GET /api/wallet/history
GET /api/rewards/history

Rewarded Ads

POST /api/rewards/ad/opportunity
POST /api/rewards/ad/verify
POST /api/rewards/ad/claim

Revive

POST /api/games/:sessionId/revive/request
POST /api/games/:sessionId/revive/verify

Double Score

POST /api/games/:sessionId/double-score/request
POST /api/games/:sessionId/double-score/verify

Double Credits

POST /api/games/:sessionId/double-credits/request
POST /api/games/:sessionId/double-credits/verify

Engagement

GET /api/daily-missions
POST /api/daily-missions/:missionId/claim

GET /api/achievements
GET /api/leaderboard
GET /api/events
GET /api/events/:eventId/progress

These routes are suggested contracts. You may adjust the route names, but all required functionality must be available and documented.

For every endpoint, document:

- Purpose.
- Authentication requirement.
- Request format.
- Response format.
- Validation rules.
- Error responses.
- Rate limits where applicable.

PART XII — AUDIO, VISUAL DESIGN, AND UX

69. Game Home Screen

Create an original, polished home screen with:

- Game title.
- Spaceship hero artwork.
- Animated cosmic background.
- Player level and XP.
- Selected ship.
- Credits and Energy Cores.
- Eligible reward balance where applicable.
- Play or Continue Mission button.
- Campaign and sector shortcut.
- Ship hangar.
- Power-up collection.
- Daily missions.
- Daily rewards.
- Achievements.
- Leaderboard.
- Settings.
- Reward center.

The screen must feel like a game interface, not a static poster.

70. AI-Generated Artwork

Generate original artwork specifically for this game.

Use a suitable image-generation tool, such as ChatGPT image generation, Google Gemini, Adobe Firefly, or another tool available to you.

Example AI prompt

Create original premium 3D sci-fi arcade game artwork for a futuristic space shooter. Feature an original compact combat spaceship flying through a deep-blue nebula, surrounded by distant alien planets, luminous asteroids, stars, subtle laser trails, and atmospheric cosmic clouds. Use cinematic lighting, detailed metallic materials, vibrant cyan and violet energy accents, strong depth, and a polished game-art style. Compose the scene for a responsive game home-screen hero background, leaving a relatively uncluttered area for frontend text and buttons. Do not include text, logos, UI controls, watermarks, recognizable copyrighted ships, or existing game characters.

Generate additional assets as necessary:

- Original player ships.
- Enemy ships.
- Boss illustrations.
- Power-up icons.
- Planet and sector backgrounds.
- Mission thumbnails.
- Ship-hangar artwork.

Do not use the AI to generate a complete screenshot containing buttons and menus. Generate the artwork separately and build all interactive interface elements in code.

Document the image-generation tool and prompts used.

71. Asset Optimization

Before deployment:

- Compress generated images.
- Use WebP or AVIF where appropriate.
- Produce suitable sizes for mobile and desktop.
- Avoid serving unnecessarily large assets.
- Lazy-load noncritical images.
- Reserve dimensions to prevent layout shifts.
- Separate backgrounds and foreground elements when this improves animation.
- Keep source assets organized.

The game must remain responsive on lower-powered mobile devices.

72. Animation and Effects

Implement appropriate effects for:

- Ship movement.
- Firing.
- Projectile impact.
- Enemy destruction.
- Boss attacks.
- Shield activation.
- Power-up activation.
- Pickup collection.
- Mission completion.
- Reward granting.

- Level-up notifications.
- Ship selection and upgrades.

Use particles and screen effects carefully. Avoid excessive visual clutter that hides enemy projectiles or makes the game unfair.

Support reduced-motion preferences where appropriate.

73. Audio — Mandatory

Provide appropriate audio feedback for:

- UI clicks.
- Ship firing.
- Projectile impact.
- Enemy destruction.
- Boss appearance.
- Boss attacks.
- Shield activation.
- Power-up activation.
- Pickup collection.
- Mission victory.
- Player defeat.
- Reward confirmation.
- Achievement unlock.

Provide separate controls for:

- Background music.
- Sound effects.
- Master volume where appropriate.

Persist supported audio preferences.

Use original or appropriately licensed audio. Handle browser autoplay restrictions and allow audio to start after user interaction.

PART XIII — ADMINISTRATION AND ANALYTICS

74. Reward and Economy Audit

Administrators should be able to inspect:

- Rewarded-ad opportunities.
- Completed ads.
- Failed ads.
- Rejected rewards.
- Credits granted.
- Credits spent.
- Revive requests.
- Revive completions.
- Double-score requests.
- Double-Credits requests.
- Power-up reward usage.
- Average rewards per user.

Restrict administrative access using proper authorization.

75. Economy Dashboard — Recommended

Build an admin economy dashboard displaying:

- Total ad opportunities.
- Completed rewarded ads.
- Rewarded-ad completion rate.
- Credits granted.
- Credits spent.
- Average reward per user.
- Revive conversion.
- Double-score conversion.
- Double-Credits conversion.
- Reward rejection count.

Metrics should be calculated from recorded events, not fabricated demonstration values.

76. Analytics Events

Track relevant events such as:

game_started
mission_started
mission_completed
mission_failed
boss_defeated
ship_selected
ship_upgraded
powerup_used
revive_requested
revive_completed
double_score_requested
double_score_completed
double_credits_requested
double_credits_completed
ad_opportunity_created
ad_completed
reward_granted
reward_rejected
daily_mission_completed
achievement_unlocked

Record only the information needed for product analytics, debugging, and economy management.

77. Economy Metrics

Design the system so it can calculate:

- Rewarded-ad completion rate.
- Revive conversion rate.
- Double-score conversion rate.
- Double-Credits conversion rate.
- Average rewards per player.
- Currency sources and sinks.
- Average mission duration.
- Missions completed per session.
- Player progression.
- Power-up usage.

Use these measurements to balance the game and economy rather than increasing advertisement frequency blindly.

PART XIV — TESTING

78. Mandatory Gameplay Tests

Verify:

1. The player can move and fire.
2. Enemies spawn according to mission configuration.
3. Projectiles cause the correct damage.
4. Enemy destruction works.
5. Boss health and phases work.
6. Power-ups have their intended effects.
7. Player defeat is handled correctly.
8. Mission completion conditions are enforced.
9. Restart creates a valid new run.
10. Mission progression is persistent.
11. Ship selection and upgrades work.
12. Audio settings work independently.
13. Mobile and desktop controls work.

79. Mandatory Economy Tests

Verify:

1. A player cannot spend more currency than they own.
2. A failed purchase does not deduct currency.
3. Duplicate reward callbacks do not grant duplicate rewards.
4. Expired ad opportunities cannot be claimed.
5. An unverified advertisement does not grant a reward.
6. A player cannot claim a daily reward repeatedly.
7. Mission rewards cannot be claimed multiple times.
8. Double-score limits are enforced.
9. Double-Credits limits are enforced.
10. Revive limits are enforced.
11. Unauthorized users cannot alter another player's balance.
12. Invalid mission results do not grant rewards.
13. Wallet and reward ledgers remain consistent.
14. Daily and session caps are enforced.
15. Cooldowns are enforced.
16. A failed ad does not consume a reward opportunity improperly.
17. Power-up purchases cannot create negative balances.
18. A completed session cannot be reused for new completion rewards.

80. Security Tests

Test:

- Authentication.
- Protected routes.
- Input validation.
- Rate limiting.
- Authorization.
- Session ownership.

- Reward idempotency.
- Duplicate request handling.
- Invalid transaction handling.
- Environment-secret protection.
- Database constraints.

Use automated unit and integration tests for critical backend logic.

PART XV — DEPLOYMENT AND DOCUMENTATION

81. Suggested Repository Structure

```
space-shooter-odyssey/
  client/
    src/
      components/
      game/
      scenes/
      pages/
      ships/
      powerups/
      economy/
      services/
      config/
      hooks/
      utils/
    public/
      assets/
        ships/
        enemies/
        bosses/
        backgrounds/
        powerups/
        audio/

  server/
    src/
      config/
      controllers/
      models/
      routes/
      middleware/
      services/
        ads/
        economy/
        game/
      validators/
      tests/

  README.md
  .gitignore
  .env.example
```

This is a suggested structure. Modify it as necessary while keeping gameplay, UI, economy, and backend responsibilities separated.

82. GitHub Requirements

Submit a GitHub repository containing:

- Complete frontend.
- Complete backend.
- Game configuration.
- Economy configuration.
- Mission and enemy data.
- Tests.
- .env.example.
- README.
- Setup instructions.

Do not commit:

- .env files containing secrets.
- Database credentials.
- JWT secrets.
- Ad-network secrets.
- Private keys.
- Unnecessary build artifacts.
- node_modules.

83. README Requirements

The README must explain:

1. Project overview.
2. Reference game.
3. Original design decisions.
4. Gameplay and controls.
5. Mission and sector progression.
6. Ship classes and upgrades.
7. Power-up mechanics.
8. Economy design.
9. Rewarded-ad placements.
10. Reward verification.
11. Wallet and transaction ledger.
12. Database structure.
13. Authentication.
14. Game session lifecycle.
15. Anti-cheat limitations and protections.
16. API documentation.
17. Environment variables.
18. Local setup.
19. Test commands.
20. Deployment.
21. Screenshots.
22. Demo video.
23. Known limitations.

84. Economy Documentation — Mandatory

Explain:

- How Credits are earned.
- How Energy Cores are earned.
- How XP is calculated.
- How platform-eligible rewards are handled.
- How players spend currency.
- Mission and boss reward rules.
- Daily mission rewards.
- Daily streak rewards.
- Rewarded-ad opportunities.
- Reward verification.
- Reward caps.
- Revive limits.
- Double-score limits.

- Double-Credits limits.
- Cooldowns.
- Daily and session limits.
- Wallet transactions.
- Idempotency.
- Server-side validation.
- Advertising and reward sustainability.

The documentation must explain the difference between in-game currency and any separately controlled platform reward currency.

85. Deployment Requirements

Submit:

- A working frontend deployment.
- A working backend deployment.
- A configured database.
- Correct production environment variables.
- A GitHub repository.
- .env.example.
- A README.
- A demonstration video.

The deployed application must actually work. A successful build alone is not sufficient.

PART XVI — DEMONSTRATION AND ACCEPTANCE

86. Demo Video — Mandatory

The demo video must demonstrate the following.

Gameplay

1. Landing page.
2. Player registration or login.
3. Player dashboard.
4. Ship hangar.
5. Ship selection.
6. Ship upgrade.
7. Mission selection.
8. Ship movement.
9. Firing and enemy destruction.
10. Power-up activation.
11. Boss battle.
12. Mission completion.
13. Player defeat.
14. Revive offer.
15. Restart mission.
16. Mission progression.

Economy

17. Wallet balance.
18. Mission reward.
19. Double-score offer.
20. Double-Credits offer.

21. Rewarded-ad demonstration.
22. Reward verification outcome.
23. Reward history.
24. Currency spending.
25. Reward limits.
26. Daily mission.
27. Achievement.
28. Leaderboard.

Backend and Reliability

29. Persistent player progress.
30. Database records.
31. Game-session handling.
32. Server-side validation.
33. Duplicate reward rejection.
34. Environment configuration.
35. Responsive mobile gameplay.
36. Desktop gameplay.

Clearly label simulated advertisements if a real provider has not been integrated.

87. Mandatory Features Checklist

Game

- [ ] Reference research completed.
- [ ] Original game identity.
- [ ] Dynamic .env game name.
- [ ] Centralized game configuration.
- [ ] Playable combat.
- [ ] Responsive player controls.
- [ ] Enemy system.
- [ ] Enemy waves.
- [ ] Five original boss encounters.
- [ ] Multiple sectors.
- [ ] 50 playable missions.
- [ ] Mission objectives.
- [ ] Mission unlocking.
- [ ] Mission result screen.
- [ ] Score and star system.
- [ ] Ship hangar.
- [ ] Five ship classes.
- [ ] Ship upgrades.
- [ ] Eight required power-ups.
- [ ] Pause, restart, and resume.
- [ ] Audio controls.
- [ ] Responsive design.

- [ ] Optimized original assets.

Full Stack

- [ ] Authentication.
- [ ] Player dashboard.
- [ ] Database persistence.
- [ ] Game sessions.
- [ ] Mission history.
- [ ] Player progress.
- [ ] Ship inventory.
- [ ] Server-side validation.
- [ ] Anti-cheat protections.
- [ ] API documentation.

VELoop Economy

- [ ] Credits.
- [ ] Energy Cores.
- [ ] XP progression.
- [ ] Separate platform-eligible rewards.
- [ ] Wallet ledger.
- [ ] Reward transaction ledger.
- [ ] Ad opportunity tracking.
- [ ] Reward verification architecture.
- [ ] Reward idempotency.
- [ ] Revive via rewarded ad.
- [ ] Double Score via rewarded ad.
- [ ] Double Credits via rewarded ad.
- [ ] Power-up reward via rewarded ad.
- [ ] Reward caps.
- [ ] Cooldowns.
- [ ] Daily and session limits.
- [ ] Reward history.
- [ ] Economy configuration.
- [ ] Server-side reward calculation.
- [ ] Economy tests.

Delivery

- [ ] GitHub repository.
- [ ] Frontend deployment.
- [ ] Backend deployment.
- [ ] .env.example.
- [ ] README.
- [ ] API documentation.
- [ ] Tests.
- [ ] Demo video.

88. Bonus Features

These are optional and should be attempted after all mandatory requirements work:

- Additional playable missions.
- Procedural mission generation.
- Ship cosmetics.
- More boss phases.
- Weekly and monthly leaderboards.
- Friend leaderboards.
- Cloud save improvements.
- PWA support.
- Offline practice mode.
- Seasonal events.
- Advanced analytics.
- Remote economy configuration.
- Multiple advertising-provider adapters.
- Reward fraud detection improvements.
- Admin mission editor.
- Admin reward controls.
- A/B testing for economy values.

Do not prioritize bonus features over broken core gameplay or reward integrity.

89. Final Quality Standard

This is an advanced full-stack internship assignment.

The evaluator will assess:

- Originality.
- Gameplay quality.
- Technical architecture.
- Maintainability.
- Responsiveness.
- Power-up implementation.
- Mission progression.
- Economy balance.
- Reward verification.
- Security.
- Testing.
- Documentation.

- Deployment quality.

A game with attractive screens but no functioning combat is incomplete. A reward system that grants currency on a button click is incomplete. A leaderboard that accepts arbitrary scores is incomplete. A wallet without a transaction history is incomplete.

Every mandatory feature must be implemented, tested, documented, and demonstrated.

90. Final Instruction

Study the reference game, develop an original space-shooter experience, and build a secure, extensible rewards economy suitable for VELoop Rewards.

Prioritize enjoyable gameplay, clear progression, meaningful optional rewarded-ad opportunities, controlled currency distribution, reliable server-side validation, and high-quality engineering.

The reference game is an inspiration for genre research—not a template to copy.

Expected outcome: A deployable, playable, full-stack space-shooter game with original missions, spaceships, enemies, bosses, power-ups, persistent progression, and a properly controlled rewarded-ad economy.